

Práctica 03: Manejo de Errores por Capas

Objetivos

- Crear jerarquía de excepciones personalizadas
- Implementar exception handlers globales
- Traducir errores técnicos a errores de dominio
- Producir respuestas de error consistentes

Descripción

En esta práctica implementarás un sistema completo de manejo de errores donde cada capa tiene sus propias excepciones y los handlers globales las traducen a respuestas HTTP apropiadas.

Estructura

```
starter/
├─ main.py
├─ database.py
├─ exceptions/
│  └─ __init__.py
│  └─ base.py          # Excepciones base
│  └─ product.py       # Excepciones de Product
├─ handlers/
│  └─ exception_handlers.py # Handlers globales
├─ models/
│  └─ product.py
├─ schemas/
│  └─ product.py
│  └─ error.py          # Schema de error
├─ repositories/
│  └─ product.py        # Traduce errores de DB
├─ services/
│  └─ product.py        # Lanza excepciones de dominio
└─ routers/
   └─ products.py       # Sin try/except (handlers globales)
```

Pasos

Paso 1: Excepciones Base

Abre `starter/exceptions/base.py` y descomenta la jerarquía de excepciones:

- `AppException` - Base
- `NotFoundError` - 404
- `ConflictError` - 409
- `ValidationError` - 400

Paso 2: Excepciones de Dominio

Abre `starter/exceptions/product.py` y descomenta:

- `ProductNotFoundError`
- `ProductAlreadyExistsError`

- `InsufficientStockError`

Paso 3: Schema de Error

Abre `starter/schemas/error.py` y descomenta `ErrorResponse`.

Paso 4: Exception Handlers

Abre `starter/handlers/exception_handlers.py` y descomenta los handlers globales.

Paso 5: Repository con Traducción

Abre `starter/repositories/product.py` y observa cómo traduce `IntegrityError` a excepciones de dominio.

Paso 6: Service con Excepciones

Abre `starter/services/product.py` y observa cómo lanza excepciones específicas.

Paso 7: Router Limpio

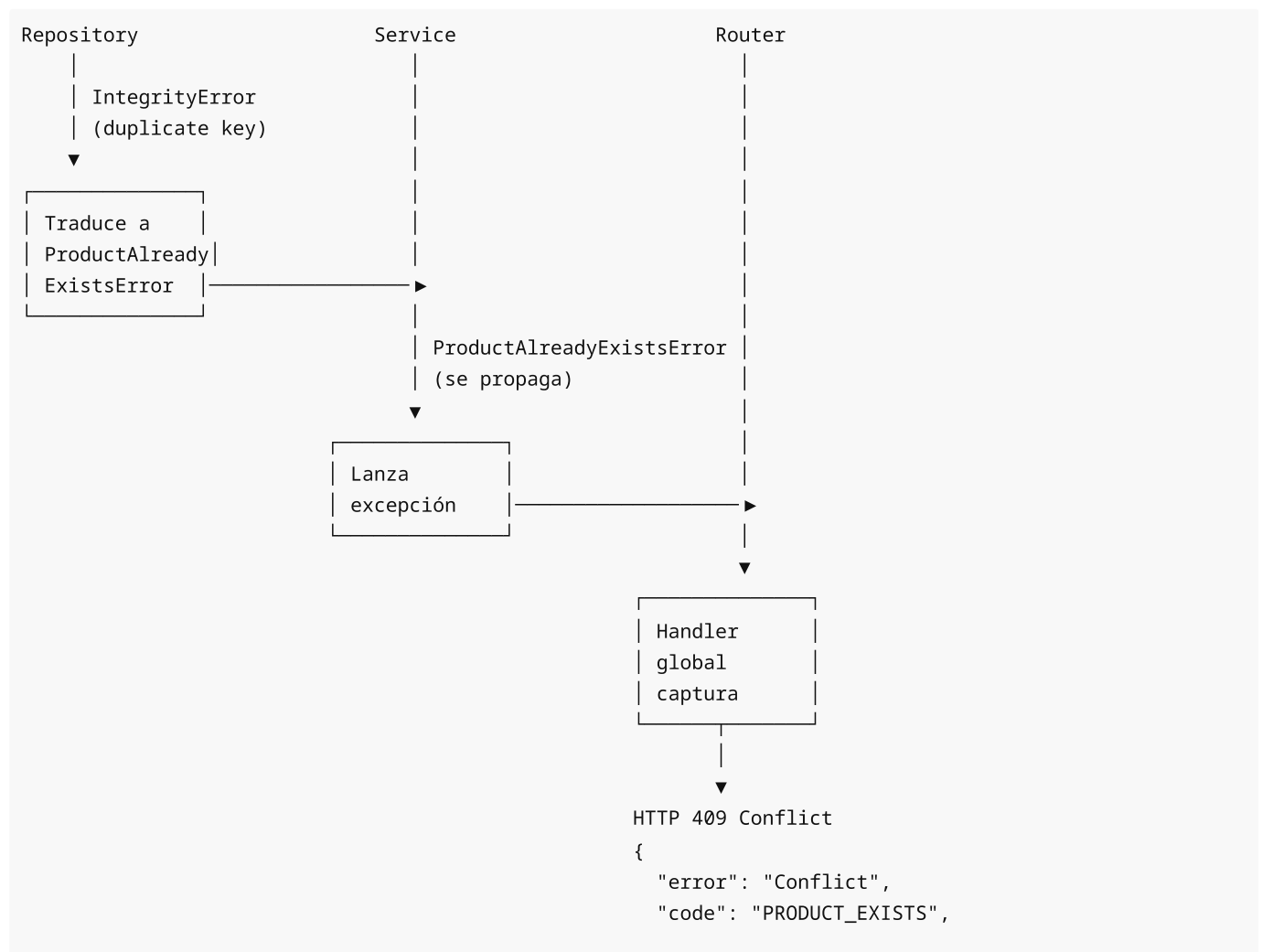
Abre `starter/routers/products.py` y observa cómo NO tiene try/except (los handlers globales manejan todo).

Paso 8: Registrar Handlers

Abre `starter/main.py` y descomenta el registro de handlers.



Flujo de Errores



```
"detail": "..."  
}
```

✓ Verificación

```
cd starter  
uvicorn main:app --reload
```

Prueba estos escenarios:

1. **GET /products/99999** → 404 con formato consistente
2. **POST /products/** (SKU duplicado) → 409 con código de error
3. **PATCH /products/{id}/reduce-stock** (sin stock) → 400

Todas las respuestas de error deben tener el mismo formato:

```
{  
  "error": "Not Found",  
  "code": "PRODUCT_NOT_FOUND",  
  "detail": "Product with id 99999 not found"  
}
```